DATE	ACTIVITY LOG
04/06/25	I have completely changed the EPQ project idea. I am no longer doing “Potential dangers of AI - especially when paired with the unmatched processing power of quantum computers.” Instead, I will now be making an AI maze solver. This is an artefact instead of a dissertation. I am making this change because I believe I will be more interested in making an artefact over making a dissertation. Furthermore, I believe that actually making a piece of software will be more impressive for university than just writing about AI.
06/06/25	Began filling out my EPQ project proposal form with the working title and reasons I want to do this topic. At this point I began to plan how I was actually going to approach this daunting task by having a look at my further maths and computer science lectures. I also considered the programming language that I was going to use for my artefact and decided to use python because currently this language is the most well known programming language for AI in industry. This ensures that the code will be built on a strong and largely relevant technological foundation.
06/06/25	Added research into the algorithms I am doing to my research sources form. I added 3 algorithms and all 3 I got from Peter Symonds on the behalf of exam boards such as AQA and OCR. The first algorithm is a breadth first graph traversal algorithm. This is to ensure that the randomly generated maze is connected so it is solvable. The second algorithm I have is Prim’s algorithm to help turn the meaningless graph into a maze. The final algorithm is Dijkstra’s algorithm which will mathematically solve the maze. These algorithms I will describe in more detail in the project though. Using academic sources helped me to define a reasonable technological depth for my project, which would make the coding of the artefact a much more straightforward process.
08/06/25	Added research into the libraries I will be using to my research sources. Libraries in programming are bundles of classes with various methods and attributes that provide additional functionality to my program. This is code that someone else has written, but this project would not be possible to do within this timescale if I were coding everything from scratch in vanilla python. The libraries I will be using are NumPy for matrix manipulation; Gymnasium for AI training and testing; Seaborn for visualisation of statistics and pygame for the final easily digestible UI.
09/06/25	Added research into maze theory/game theory I will be using to my research sources. Graph theory is one branch of mathematics that revolves around graphs. These are not your standard cartesian graphs which have (x,y) coordinates but are a set of points (called nodes) connected by lines (called arcs.) Graphs can be used to represent a variety of complex relationships such as connections between users in a large database. By adding weights to each arc on the graph, you produce a network, which also has multiple applications (e.g. neural networks) but in my case, a network is what I will be using to represent the maze. I needed to ensure that I conducted this research into great depths because it underpins the entirety of my project.
11/06/25	Created a searching for info document and added my methods of researching into sources. Peter Symonds College has provided a variety of both paid and free methods of researching into this topic. These include: - Gemini (LLM) for general research and ideas. - MyBib/Cite Them Right for referencing. - Credo for in depth academic research. Gemini is particularly useful for getting a rough idea of what to do.

DATE	ACTIVITY LOG
20/06/25	I began writing my research sources. This is an in depth analysis of all the algorithms I will be using and how they work as well as the theory behind each algorithm. This is also where I will go into greater depth into my Q-Function. This was not an overly difficult task because I had already been taught a lot of graph theory in both further maths and computer science. Alongside this, I felt it was appropriate to manually draw out examples of certain aspects of graph theory so that I can better demonstrate my point.
24/06/25	Added more research to my research sources. This was research into how a queue works. A queue is important for my breadth first traversal, which will be a key algorithm in my maze generation techniques. I then proceeded to do an in depth analysis of both a queue and a breadth-first traversal in my research review. I have also added detail into how prim's algorithm works. Both these mean I have now fully described my maze generation technique. I have also gone into depth on how Dijkstra's algorithm works which will give me the necessary benchmarks to compare the AI against. Again, I already had a strong understanding of these algorithms due to already having learnt them in further maths and computer science. However, actually writing out how the algorithms work proved to be a much more involving challenge and helped me clear any lingering doubts about certain technicalities of each algorithm.
25/06/25	Added research into Reinforcement learning into my research review. This involved watching the video that I found as well as reading into the website I found. I also added research into deep-q-functions into my research sources so I could roughly explain why I chose against using them. Also referenced my sources inside the research review. This was an incredibly difficult challenge because I had very little knowledge on AI prior to this project. I had to watch the youtube video multiple times alongside the website that I used to fully understand what was going on, before I could even begin to describe what was happening in detail.
26/06/25	Added an additional bit of research into my reinforcement learning inside my research review. This was about the epsilon greedy policy which I didn't fully understand as it was only briefly mentioned in the video. This also changed the way I believed I was going to initialise my tabular Q-Function. Before understanding this epsilon greedy policy I was planning on filling my Q-Function with a bunch of random Q-Values and I assumed that the Bellman equation would have adjusted these values to become correct. This is an incredibly inefficient way of performing the training and so I am happy that I went deeper into this research. Using epsilon now allows me to have an initial Q-Function full of state action pairs, all with a Q-Value of 0, and allows the agent to perform exploration steps.

DATE	ACTIVITY LOG
27/07/25-10/07/25	There were many open days and university lectures so I was less available over this time period to work on the project. I did look on the internet to find some exemplar EPQ artefacts where people coded something just to have a rough idea of what the artefact should look like. There weren't very many I could find however, I did find one decent scoring project where the person developed a video game in Unity. I believe the biggest learning outcome of this project was creating achievable goals. The person's code indicated that they were incredibly capable of developing a high quality program, however, due to their unrealistic goals, they were unable to fully complete the program to the level that they wanted to achieve. This has reinforced my decision to not create a deep-q-learning AI model and to stick to a standard tabular q-function as I don't think it is feasible to create such a complex model.
15/07/25C	I began programming on this day. I started off by opening up the IDE and just making some very short and quick projects to recap some of the syntax. I recapped selection statements; loops and classes because these were going to be a major part of the actual project. I also opened up one of my childhood pet projects where I built a space invaders clone using pygame. This is so that I could recap the overall structure of the code when using pygame and some of the more important subroutines that I might have to use when developing the project. Fortunately, python is a high-level coding language and a lot of the syntax is quite similar to written english. Therefore, switching from C#, which is more heavily typed, was a fairly comfortable change.
15/07/25	On this day I also created the maze solver file and began making the testing 2D arrays. I did this by actually drawing out graphs on a sheet of paper that I deemed would cause a certain unique result for each of the different algorithms. I also wrote the breadth-first-traversal algorithm on this day. Because of the in depth research I had done beforehand, I was able to make this algorithm work after with very little difficulty. At this stage in my EPQ I had to make sure that I followed the plan in the project proposal form incredibly closely because I also had many other upcoming commitments such as the TMUA examination. I wanted to get the program done a couple weeks before the exam so that I could spend more time revising for the exam rather than programming.
17/07/25	Today I finished coding Prim's algorithm to convert a maze into a minimum spanning tree. Similar to the breadth-first-traversal, I was able to quickly get this algorithm working because of my strong research into the theory behind this algorithm. I also created some functions to actually perform the random generation of the maze. These algorithms should have been much easier to create than Prim's algorithm however, in actually generating a random graph I made a fundamental error. Because the graphs I am using are undirected graphs, their adjacency matrix's should be symmetrical on the leading diagonal. However, my initial random graph generating function gave a random number for every entry in the matrix, meaning the matrix was not symmetrical down the leading diagonal. This led Prim's algorithm to fail and so I spent about 1h debugging and trying to fix Prim's algorithm, only to realise the mistake was with the random directed graph. Although this mistake was time-consuming, it reinforced the idea that I should debug

DATE	ACTIVITY LOG
17/07/25C	much more systematically and double check all aspects of my program. Fortunately, I caught this mistake quite early on as it would have been difficult to spot in the later stages of my program.
17/07/25	After I had finished up the maze-generation, I worked on the algorithmic maze solving. I found Dijkstra's algorithm slightly harder to implement than the previous algorithms because it needed to be done in two stages, however due to further maths, I already had a lot of experience with actually carrying out Dijkstra's algorithm so actually implementing it was not too difficult. Generally, when coding I like to create my own code rather than using the internet as I understand my own code better than what someone else created. Therefore, I created this algorithm based on my knowledge of further maths. After creating both the forward and the backward pass subroutines I did have a look at my computer science lesson notes. We have not covered Dijkstra's in computer science yet, however the notes revealed that there was an easier way to code the algorithm without needing a backwards pass, had I just stored the previous fixed node for all updated neighbours. I have now learnt that I always need to look at a broad set of sources during research as this could have also saved me quite a lot of time (~1h for the backward pass).

23/07/25	Today I created the RL AI. I started by actually creating the AI class containing some constants and attributes that I thought were necessary for the training. I also rewatched the video on how the tabular q-function works to be certain of the whole process before actually implementing it in practice. I created some highly useful quality of life subroutines that greatly simplified the programming process. I then actually began coding the training function. Counterintuitively, I ended up coding the procedure in reverse, starting with each step, followed by each episode and then finally stitching the episodes together in a loop. I made a couple errors while coding, one of the most notable ones being my initial incorrect usage of the Bellman equation. As shown in my research review the Bellman equation is:Q(s,a) = Q(s,a) + alpha*(r + gamma*max(Q(s',a')) - Q(s,a))
	The most important part to note is that inside the gamma*max there is s',a'. This means to find the maximum discounted Q-function value of the NEXT state. I misread the equation and did not copy over the apostrophes, and so I initially implemented the operation finding the maximum discounted Q-function value of the CURRENT state. This is clearly incorrect and the training did not work because of this. After some debugging I found this error and corrected it and the training cycle ended up working (I compared the path found during training to the path found using dijkstras.)
	
	Here is the broken code:
	`row[2] = row[2] + self.CONST_LEARNINGRATE*(-self.mazeToSolve[currentNode][nextNode] + self.CONST_DISCOUNTFACTOR*self.getNodeWithHighestQFunctionValue(self.tabularQFunction, currentNode)[0] - row[2])`
	
	Here is the fixed code:
	`row[2] = row[2] + self.CONST_LEARNINGRATE*(-self.mazeToSolve[currentNode][nextNode] + self.CONST_DISCOUNTFACTOR*self.getNodeWithHighestQFunctionValue(self.tabularQFunction, nextNode)[0] - row[2])`
	
	I think this mistake really highlighted the importance of attention to detail as such a small mistake of not copying over an apostrophe caused the whole training not to work. This was the stage that I was most relieved with in the entire project as I was unsure whether or not it was actually going to work. Initially, going into this project, I assumed that I was going to use some python libraries to actually make the AI but I ended up coding this in vanilla python which was very

DATE	ACTIVITY LOG
23/07/25C	rewarding. Below the training function I also added the testing function however this was a relatively quick addition because the two functions were closely related in their structure and behaviour.
23/07/25-05/08/25	Abroad
06/08/25	Today I began planning how to actually create my user interface. Initially I was drawn towards using pygame as I had already used pygame before and was familiar with the syntax. Therefore, I opened up a new project and just began experimenting with the library. I tried to manually create some of the common objects found in graph theory such as a node or an edge, however I found it extremely difficult. The reason for this was that pygame wasn't designed for discrete mathematics and operated on standard cartesian coordinates. At this stage I realised that if I were to create this project in pygame I would need to start and build everything from scratch. Not only would this be a massive undertaking but it also meant that I wouldn't be able to create a lot of the necessary features such as a planarity check. This would significantly worsen the quality of my project, so I decided against pygame. Since I didn't know any other visualisation libraries I went to Gemini and explained what I would actually need for my project. Gemini suggested that I have a look at the documentation for the libraries networkX and matplotlib. After scanning the documentation, I found that these two libraries were indeed more suitable for my project. I downloaded them onto my computer and began playing around with some of the graphs and features of the library. These libraries enabled me to easily draw graphs on the screen. networkX came with a built in planarity check, which would be incredibly useful for higher quality visualisations. These libraries also allowed me to colour certain nodes and arcs, which would be great when demonstrating the AI training process and showing the final path. I have provided a copy of the conversation I had with Gemini to find these libraries in my research sources.
07/08/25	I began implementing the UI libraries I found yesterday into the project. I started off by creating some quality of life procedures that saved me a lot of time in the project later on. I also experimented with the type of graph layout I would be using and the x,y coordinate positions of the start and the end nodes to produce the highest quality maze. Once I was satisfied with the result I began to connect the code to colour certain nodes and arcs to both Dijkstra's algorithm and the AI training subroutines. Since the core functionality of the program had already been written, adding UI on top was not too much of an extra step. I did also add some time delays into the UI visualisation because the AI training process was too fast and meant any visualisation would occur too quickly for anyone to understand. This is the only stage that I was uncertain about because there was a major conflict of interest. From a programmatic perspective, adding time delays into an algorithm is unheard of because it completely hinders the efficiency of the program. A large portion of programming involves optimising code to try and reduce the time complexity as much as possible. However, from an EPQ perspective, adding these time delays was necessary as this is meant to be an educational piece of software.

DATE	ACTIVITY LOG
08/08/25-30/09/25	Working on my computer science NEA coursework (in C#.) I was also abroad for a couple weeks. In this time I properly began TMUA revision
01/10/25	I had a very different type of task today. My goal for the near future was to get my program from my laptop to the school PCs. To achieve this I first compiled the python code into a .exe file. This sounds counterintuitive because python is an interpreted language, however I used a bundling tool that I found online to achieve this. I then uploaded both the code and the executable to my school google drive so that I could access it from the college. When on the college PCs, I tried to download both the code and the executable. The code downloaded on the PC with very little issue however the executable was difficult. The first few times I tried to download the executable completely failed and google drive said that it was not possible to download the file. I opened the executable from google drive using .zip extractor and it downloaded on the PC successfully. However, when I tried to run the executable on the PC nothing happened. I am not 100% sure why this .exe file didn't run (or at least didn't give a warning message) but most likely what is happening is the .exe is being blocked by the PC's antivirus. I found here to be another conflict of interest, because I have a moral obligation not to do anything harmful to the school devices, however I also want to complete my EPQ and I know for certain that this .exe is not harmful to the devices because I created it. I then switched from working with the .exe to working with just the code. The school PC's do have python installed however, this version of python does not have the libraries networkX or matplotlib installed, so running the code on the PC's caused an error. I also tried to download networkX and matplotlib, but since I am unable to run executables that I downloaded from the internet on the PC's, this was pointless. My first attempt at getting this code to work on the PC's was unsuccessful.
02/10/25	Today I had a discussion with my father about how I could go about running the program on the school device. I believed my father could give me a good insight into this because he is a software developer and so he could show me potential ways to overcome these challenges. First of all, since the .exe failed silently, he suggested that the .exe for the AI Maze solver may have downloaded incorrectly or have been corrupted during transfer and he told me to go find a better way of downloading this file on the device. As for the actual program itself and downloading the libraries matplotlib and NetworkX he said it is reasonable that the school network prevented me from downloading these files as it would be a massive security concern. He suggested that I do some more research into portable environments. Given his feedback, I prepared a multitude of resources for the following day of college. Firstly, I prepared a new folder on my laptop where I would add a variety of resources. I downloaded a python portable environment containing the code for matplotlib and networkX into the folder. This program contains the tools required to make python run even when python is not installed on the device. I also put my .exe in this folder and the actual code for python. To minimise errors during transfer, I prepared two methods of data transfer. The first method was to use a USB pen drive with the folder in it. The second method was to host a server on my laptop that was available to the internet. I was able to do this using a tool I found using Ngrok.com.

DATE	ACTIVITY LOG
03/10/25	After the well thought out planning on the previous day I brought my laptop to college. I was more confident that the server plan was going to work so I started off with the server. The school network didn't let me host a server which was expected, as this was another large security breach. To overcome this I used my phone's mobile data and hotspotted my laptop. I then turned on the server on my laptop and used Ngrok to make this server available to the internet. I made sure that the folder containing the files was linked to my laptop's server. I then typed in the URL provided by Ngrok to access the server. Fortunately, I was able to see all the files in the folder that were on the laptop. I first downloaded the .exe file from the server onto the PC and attempted to run it. Sadly, the same result occurred on the first day and nothing happened. At this point I was certain that I was not going to be able to run this executable on the PC's. Then, I encountered an issue with my server downloading software and that was its inability to download entire folders - only allowing me to download singular files. Since the portable python environment consisted of thousands of files, this meant the server was not practical for getting the environment on the PC's. I then plugged in the USB pen drive to the PC's and to my surprise, I was able to move files to and from the device. This meant I was able to get the portable environment onto the device which was a huge success. However, this success was short lived as IDLE python doesn't allow you to use a custom python environment and instead uses the PC's default python environment defined in the systems ENVIRONMENT variables. Due to the constrictive nature of the PC's, I was unable to modify these variables to use the custom python environment. I also tried to run the python program through the terminal, however, the terminal has been disabled on these PCs (again for security reasons.) In a last ditch effort I created a: - Windows batch file - Windows powershell script - VBScript script file All of which would try and run the command: my_env\Scripts\python.exe AIMazeSolver.py However none of these attempts worked. This is probably for the same reason that I am not allowed to run executables that I have taken from the internet. At this point I was convinced that I was probably not going to get the program running on the PCs. Simply, the PCs have too many security features for me to be able to run the program on them. If I tried further and dug deeper I fear I would be breaching the college's software policy and so I ended my search here. The one small success I found was to run the program using google collab (an online IDE), however the UI on google collab looked poor. To demonstrate my AI in the classroom, I will bring in my laptop and connect it to the whiteboard. For any potential examiners, I have attached video recordings of the AI working on my laptop. I have also attached the full code if an examiner would like to test the code for themselves on a less restrictive device (they will however need to download python; matplotlib and networkX on their device.) I am unable to attach an executable file to google classroom for an examiner of this project. At this stage in my project, I am about a couple weeks out from the TMUA and so (as stated in the plan), I am slowly reducing my hours to work on the EPQ so that I can focus more on the TMUA.

DATE	ACTIVITY LOG
04/10/25-14/10/25	TMUA Revision.
15/10/25	I wrote up the beginning of my development report detailing my creation of the first testing graphs (adjacency matrices) and the development of my BFS. I tried to focus less on actually explaining the algorithm (as I had already done that in the research review) and more explaining the way the code works - and what part of the algorithm each line represents. I also wanted to make the demonstration of the code as visual as possible, by actually showing examples of the graphs I created as well as some of the forms the data for this graph is transformed into, however, given the nature of this project, this is difficult.
22/10/25	I continued writing my development report, this time detailing the graph generation process. I explained how my program applies Prim's algorithm to convert the graph into a minimum spanning tree and then linked this back to my graph generation process. At this point, I am finding the development description more difficult than actually coding the program. I have identified that the skill of describing a complex program is completely different to the skill of coding the program. This is a new challenge that I want to overcome and get good at, as currently my strong point in this project is my understanding of math and programming.
23/10/25	Today I had a lot of spare time so I continued working on my development report. This time I explained how my version of Dijkstra's algorithm worked. As the 26/10/25 deadline for the draft development report is approaching, I conducted a formal review of my remaining tasks. After today, I only needed to document the code for the AI as well as the code behind the user interface. This I estimated would take me about 2-3 work sessions, which I could do over the weekend. Therefore, I believe that this deadline is definitely achievable, even if it will be tight.
24/10/25	I added some more detail to my development report describing my AI training process. I described the structure of the tabular Q-function in my code as well as the implementation of some of the core constants to the training process. At this point I have done so much work around the reinforcement learning model that explaining how my code worked was a relatively straightforward process. Also I was finding the process of writing up and explaining the code was coming to me a lot more naturally. I was able to find the balance between not adding enough explanation and overexplaining every line of code. I wrote up about half of the training process but today I was quite busy as I had exams for both further maths and for computer science.
25/10/25	I finished writing up the second half of the training process as well as the testing function of the program. This concluded the second primary area of my program and left me with only UI to describe. At this rate I am certain that I will meet the 26/10/25 deadline.

DATE	ACTIVITY LOG
26/10/25	Today I described the code surrounding the user interface in my development report. This required me to give a brief summary of the libraries that I was using and then a more detailed description of how the visualisation subroutines would make use of this library. This process was slightly more difficult than my previous writeups because the code involved libraries. In previous write ups, I found the code easier to describe because it was code that I wrote - hence I was more familiar with it. However, a library is me using someone else's code, so when describing some of the library's functions, I had to go back to the documentation to verify that what I was saying was correct. Learning how to document integration of code rather than implementation was a crucial learning experience that I had not yet done in this project. I have now finished the development report, thus successfully meeting the deadline that I set for myself.
27/10/25-30/10/25	Due to the TMUA , I was slightly behind on my computer science coursework, so I had to catch up over this time.
31/10/25-06/11/25	I was given some feedback by my EPQ teacher on what sections of my EPQ to improve. This was primarily concerning my PPF and my activity log, which I had largely neglected while focusing on actually writing up the project. I spent the first 2 days just adding more detail into the project proposal form on all 4 sections. On the remaining days I added detail into my logs into the activity log. When I began, my logs were quite descriptive and although this was good, my EPQ teacher suggested making my logs more reflective in order to achieve AO4 marks. This was one of my primary focuses when updating some of my entries. I enjoyed this section of the project as I was able to document some of my thoughts around this project that were not really suitable in either the code or the development report.
12/11/25	Today I wrote my conclusion. I was largely confused by the difference between a conclusion and an evaluation - I initially thought they were the same thing. However after a bit of research, I found out that a conclusion was a brief final summary of the project whereas an evaluation is an in depth assessment of the methods and processes I went through to reach my final conclusion. Therefore, I concisely went through and summarised what I had achieved over this project. After I finished writing the conclusion I began writing the evaluation. I started off by assessing how well I achieved my initial objectives. Given the definition of an evaluation, I tried as hard as possible to be reflective of the project, alongside being descriptive.
13/11/25	I continued working on my evaluation, this time detailing my thoughts surrounding the research I went through that enabled me to actually create this project. Outside of the EPQ, today I received an invitation to do an interview at Cambridge university (2nd December). This has shifted my plans and focuses as now I need to spend the next few weeks preparing for this interview. Fortunately, I am nearing the end of my EPQ so I can afford to lose a few weeks to additional revision. This is another reason as to why I am grateful for my good planning as it enabled me to have a buffer week where I don’t need to work on my EPQ (to revise), but I am not behind on this work.

18/11/25	Today I added more detail into my evaluation, this time detailing some of my major thoughts surrounding the development process and some of the difficulties that I overcame. At this stage I enjoyed writing as it allowed me to provide the reader with insight into some of the things they can't really see in this EPQ. I was able to simply provide my thoughts and smaller sub-conclusions about different aspects of the project.
20/11/25	I finished my evaluation today by writing up some parts of the program that I would have liked to have improved on, had I had more time. A lot of these ideas were interesting and would result in a higher quality final program, however they didn't make it into the final project largely due to time constraints. Looking back, it is a shame that this project was over such a small period of only 6 months as I really would have liked to explore a lot of these concepts in much greater depth. I also wrote my abstract today. This was a quick piece of work that really is only there to help give a reader a brief summary of the project before they begin reading.
21/11/25-02/12/25	On the 21st I received an invitation to an interview at Cambridge University. I spent the next week and a couple days solely revising techniques needed to perform well in the interview. As a result, I believe my interview performance was good. I had accounted for these sorts of delays during my planning so I was again able to not do my EPQ for this time period and not fall behind largely.
03/12/25	I was aware that I had my EPQ presentation on the 10/12/25 so I dedicated today to solely working on making a slideshow for the presentation. There were a multitude of things that I had to consider such as: how long the presentation would take; how complex the presentation was or how interesting it would be for the audience. Taking these things into mind, I decided to limit my demonstrations to Dijkstra's algorithm and the AI reinforcement learning training process. I chose Dijkstra's algorithm because it is fairly easy to explain and it is relatable to audiences due to Sat Nav technology. I chose to explain the AI (in a simplified form) because this is the main focus of the project and it would be nonsensical to not explain it. However overall, I would say this was another enjoyable process as I was able to make my project appear much more lively.
04/12/25-08/12/25	Coincidentally, the computer science coursework deadline was also the 19th of December. Therefore, over these days I didn't do much EPQ work in order to focus more on this coursework. This decision was made knowing that my EPQ was basically finished and the only thing remaining to do was to add a few finishing touches and fixes.
09/12/25	Today I rehearsed the EPQ presentation for the following day. From my observation of the other people in my class doing their presentation, I found that not reading off of pre-written notes when explaining made the presentation far more engaging. Therefore, I spent this day consolidating my knowledge of the algorithms that I was planning on demonstrating and how I was going to simplify them for the uninformed class. During my interview preparation, I learned that recording yourself talking out loud when alone is a great way to overcome any fears surrounding public speaking. I thus recorded myself explaining some of the crucial concepts in this project and rewatched the videos to see some of my strong and weak points. Some of my classmates noted how nervous they were approaching the class presentation, however I think after the nerves of the interview, I felt this presentation was not going to be nearly as scary.

10/12/25	I actually carried out the presentation today where I demonstrated my AI Maze solver. I feel like the presentation went pretty well and I managed to compose myself enough to adequately explain some of the difficult underlying concepts. I asked the class some simple questions which surprisingly people engaged with and responded to - the reason I say surprisingly is because in previous presentations, when the presenter would ask a question the class generally would be silent. At the end of the presentation I also received many questions about many different aspects of the project which I felt I was able to respond to sufficiently. I also talked to some of my classmates afterwards and they said they thought my presentation was one of the stronger presentations, however I need to take this with scepticism as many of them will be telling me this to boost my confidence.
13/12/25	The final EPQ deadline is approaching rapidly so I took this day to fix up any additional weaknesses in my project. The first modification that I did today was adding my reasoning for not selecting certain algorithms into the research review. This was a process that I had already done before beginning the coding part of the project; however, I never officially documented it. I provided a thorough comparison between the DFS and the BFS as well as a thorough comparison between Kruskal's and Prim's algorithm. Fortunately, I had already found the necessary research sources and it was now just about synthesising them into a digestible format. Another issue that bugged me for a while was the inefficient implementation of Dijkstra's algorithm. In the development report, I explicitly stated that my current version of Dijkstra's algorithm was inefficient yet I did not carry out any further actions to rectify this inefficiency. I opened up the code and programmed the optimised Dijkstra's algorithm. This sounds like a lengthy and tedious process however, the optimised Dijkstra's algorithm was nearly identical to the forward pass of my original program, so this process only took about 10 minutes. I also filled out the candidate record sheet with relevant information about my EPQ as this was one of the official documents we needed to submit and I needed to get my EPQ teacher to sign it. As a final check I read through my EPQ just to spot out any minor spelling/technical errors that I could have made. These small issues I don't believe would fundamentally undermine the quality of my EPQ, however it felt necessary to solve them for overall completeness to my project.